

BuMoChi インストール

BuMoChi

概要

BuMoChi は複数のアプリケーションを接続します。個別にインストールするものと、BuMoChi リポジトリに含まれているものがあります。

コンポーネント	入手先	必須か
XR-Animator	公式リリースページからダウンロード	Web カメラによるモーションキャプチャーには必須。 クリップだけを扱う場合は任意
Python 3	Python 公式サイトまたは OS のパッケージマネージャーからインストール	エンコーダーとデコーダーに必須
BunrakuOSCEncoder OscGroupClient	PipelineApplications/ に同梱 HelperAppsAndExamples/OSCGroups/ 以下にバイナリを同梱	XR-Animator の入力に必須 OSCGroups を介した共同作業にのみ必須
SuperCollider	公式サイトからダウンロード	必須
BuMoChi SuperCollider ライブラリ	このリポジトリ	必須
BunrakuOSCDecoder	PipelineApplications/ に同梱	BuMoChi のアニメーションを Godot に送信するために必須
Godot	公式サイトからダウンロード	アバターのレンダリングに必須
BuMoChi Godot プロジェクト	完全版 ICLC27 ワークスペースに同梱。参照用プロジェクトは BuMoChi にも同梱	アバターを画面に表示するために必須

準備済みプロジェクトは Godot 4.5 で開発およびテストされています。再現性のある結果を得るため、Godot 4.5 をインストールしてください。動作中のパフォーマンスプロジェクトを新しい Godot バージョンへ移行する場合は、必ず先にコピーを作成してください。

特に記載がない限り、以下のパスはすべて BuMoChi リポジトリのルートを基準とした相対パスです。

BuMoChi を入手する

まだリポジトリを取得していない場合は、次のコマンドでクローンします。

```
git clone https://github.com/iani/BuMoChi.git
cd BuMoChi
```

完全版 ICLC27 ワークスペースを使用している場合、BuMoChi リポジトリはすでに次の場所にあります。

AppsAndCode/BuMoChi/

リポジトリは恒久的な場所に置いてください。以下で説明するオプションのコマンドラインランチャーと SuperCollider のインストールでは、この場所を指すシンボリックリンクを使用できます。

XR-Animator

XR-Animator は Web カメラから身体と顔の動きを取得し、VMC として送信します。

1. [XR-Animator 公式リリースページ](#)を開きます。
2. 使用している OS とプロセッサに適したネイティブデスクトップ版をダウンロードします。
3. そのリリースに付属するインストールノートを読んでください。macOS 版は現在、開発者によってベータ版と説明されています。
4. アプリケーションを BuMoChi リポジトリの外にある通常のアプリケーションフォルダーへ展開またはインストールします。
5. XR-Animator を起動し、macOS または OS から求められた場合はカメラへのアクセスを許可します。

BuMoChi パイプラインではネイティブデスクトップアプリケーションを使用してください。ブラウザー版は実験には便利ですが、ここではネイティブ VMC 出力が必要です。

この時点ではポートを設定しないでください。完全な起動手順とポート設定については、[BuMoChi フルセットアップ手順](#)および [Port Number Setup](#)を参照してください。

Python 3

2つの Bunraku パイプラインアプリケーションには Python 3 が必要です。使用するのは Python 標準ライブラリだけなので、**pip** によるインストールやサードパーティ製 Python パッケージは不要です。

Python がすでに利用できるか確認します。

```
python3 --version
```

コマンドを利用できない場合は、[python.org](#) または OS のパッケージマネージャーから Python 3 をインストールしてください。その後、新しいターミナルを開き、もう一度バージョンを確認します。

BunrakuOSCEncoder

BunrakuOSCEncoder は BuMoChi に含まれています。XR-Animator が送信する VMC バンドルを、SuperCollider と OSCGroups が使用する完全なルート情報を持たない Bunraku フレームメッセージへ変換します。

エン트리ポイントとサポートモジュールは次の場所にまとめて保存されています。

PipelineApplications/

別途インストールする必要はありません。ターミナルを BuMoChi リポジトリのルートに置き、起動できることを確認します。

```
python3 PipelineApplications/BunrakuOSCEncoder.py --help
```

BunrakuOSCEncoder.py は同じ場所にあるモジュールをインポートするため、このファイルだけをコピーしないでください。どのターミナルディレクトリからでも実行できるようにするには、[任意のターミナルディレクトリから BunrakuOSCEncoder を利用可能にする](#)で説明するオプションのシンボリックリンクランチャーを作成します。

目的、オプション、起動コマンドについては、[BunrakuOSCEncoder](#)を参照してください。

OscGroupClient

OscGroupClient が必要なのは、ワークステーション間でライブモーションソースを共有する場合だけです。単一ユーザーのローカルパイプラインでは省略できます。

BuMoChi には macOS、Linux x86-64、Windows 用のクライアントバイナリが含まれています。

```
HelperAppsAndExamples/OSCGroups/bin/macos/OscGroupClient
HelperAppsAndExamples/OSCGroups/bin/linux/arch/OscGroupClient
HelperAppsAndExamples/OSCGroups/bin/windows/OscGroupClient.exe
```

同梱の macOS バイナリは Intel **x86_64** 実行ファイルです。Apple シリコン搭載 Mac では、初回実行時に macOS から Rosetta のインストールを求められる場合があります。

macOS または Linux では、必要に応じて選択したバイナリに実行権限を付与します。macOS の場合：

```
chmod u+x HelperAppsAndExamples/OSCGroups/bin/macos/OscGroupClient
```

クライアントには OSCGroups サーバーのアドレス、ユーザー認証情報、グループ名、グループパスワードが必要です。共同作業サーバーの運用者から取得してください。

グローバルに利用できるコマンドとしてのインストール方法と、起動時に必要な 9 つの引数については、[OscGroupClient](#)を参照してください。

上流の OSCGroups ソースプロジェクトは [Ross Bencina の OSCGroups リポジトリ](#)で管理されています。

SuperCollider

BuMoChi は SuperCollider の言語環境で動作します。

1. [SuperCollider 公式ダウンロードページ](#)から、使用しているコンピュータに適した最新の安定版をダウンロードします。
2. SuperCollider IDE をインストールして起動します。
3. SuperCollider ドキュメントで次の行を評価し、ユーザー用クラスライブラリを置くディレクトリを確認します。

```
Platform.userExtensionDir.postln;
```

返されるパスはプラットフォームとユーザーによって異なります。固定パスを仮定せず、表示された値を使用してください。

BuMoChi のモーション処理には SuperCollider オーディオサーバーの起動は不要です。パフォーマンスで音声の合成または処理も行う場合にのみ、オーディオサーバーを起動してください。

BuMoChi SuperCollider ライブラリ

SuperCollider は **Platform.userExtensionDir** の下で BuMoChi リポジトリを見つけられる必要があります。次のいずれかの方法を使用します。

1. BuMoChi リポジトリフォルダー全体をユーザー拡張ディレクトリにコピーします。
2. より適切な方法として、ユーザー拡張ディレクトリ内に、恒久的な BuMoChi リポジトリを指すシンボリックリンクを作成します。これにより、作業中の Git チェックアウトとインストール済みクラスライブラリが同期した状態に保たれます。

macOS では、**Platform.userExtensionDir** が返したパスを確認した後、通常は次のようにシンボリックリンクを作成します。

```
mkdir -p "$HOME/Library/Application Support/SuperCollider/Extensions"
ln -s "/absolute/path/to/BuMoChi" "$HOME/Library/Application
↳ Support/SuperCollider/Extensions/BuMoChi"
```

/absolute/path/to/BuMoChi を実際のリポジトリパスに置き換えてください。リンク先に **BuMoChi** という項目がすでに存在する場合は、置き換える前に内容を確認してください。

リポジトリをコピーまたはリンクした後：

1. SuperCollider で **Language → Recompile Class Library** を選択します。
2. ポストウィンドウにコンパイルエラーがないか確認します。
3. 次を評価します。

```
Bmc.status;
```

有効なステータス応答が返れば、**Bmc** クラスは正しくインストールされています。クラスライブラリのコンパイルに成功すると、**Bmc** は UDP ポート **57130** でデフォルトの OSC ディスパッチャーも起動します。

BunrakuOSCDecoder

BunrakuOSCDecoder は BuMoChi に含まれています。SuperCollider/Bmc から送られるルート付きフレームを、Godot 用の標準 VMC バンドルへ戻します。

ターミナルを BuMoChi リポジトリのルートに置き、起動できることを確認します。

```
python3 PipelineApplications/BunrakuOSCDecoder.py --help
```

エンコーダーと同様に、エントリーポイントはサポートモジュールと同じ場所に置いてください。Python ファイルだけをコピーしないでください。どのターミナルディレクトリからでも実行できるようにするには、[任意のターミナルディレクトリから BunrakuOSCDecoder を利用可能にする](#)で説明するオプションのシンボリックリンクランチャーを作成します。

目的、オプション、起動コマンドについては、[BunrakuOSCDecoder](#)を参照してください。

Godot

Godot はローカルで合成されたアバターシーンをレンダリングします。

1. [Godot 4.5 公式アーカイブページ](#)から Godot 4.5 をダウンロードします。

2. 使用している OS 向けの標準ビルドを選択します。準備済みプロジェクトは GDScript を使用しており、.NET ビルドは不要です。
3. Godot をインストールまたは展開し、Project Manager を起動します。

現在のプロジェクトは Godot 4.5 との互換性を宣言しています。新しい Godot リリースでは、プロジェクトのアップグレードを求められる場合があります。パフォーマンス用途では、動作が確認された 4.5 を維持し、アップグレードは複製したプロジェクトでのみテストしてください。

BuMoChi 用 Godot プロジェクト

完全版 ICLC27 ワークスペースがある場合、準備済み Godot プロジェクトは BuMoChi と並んで次の場所に保存されています。

AppsAndCode/GodotProjects/

デフォルトの単一アバター Ishidomaru パイプラインには、次を使用します。

AppsAndCode/GodotProjects/Seed_2_Ishidomaru_C/project.godot

Mother と Ishidomaru を均一に扱う 2 体構成には、次を使用します。

AppsAndCode/GodotProjects/Seed_4_Mother_Ishidomaru_E/project.godot

プロジェクトを Godot にインポートします。

1. Godot Project Manager を開きます。
2. **Import** をクリックします。
3. プロジェクトの **project.godot** ファイルを選択します。
4. プロジェクトパスを確認し、インポートします。
5. シーンを実行する前に、Godot の初回インポートが完了するまで待ちます。

準備済みの各アバタープロジェクトには、必要な Godot VMC Tracker、VRM importer、MToon shader アドオンが、それぞれの **addons/** ディレクトリ内に含まれています。そのプロジェクトへ Asset Library から重複するアドオンをインストールしないでください。

BuMoChi の単独チェックアウトにも、身体トラッキングの参照プロジェクトが含まれています。

PipelineApplications/GodotVMCReference/project.godot

この参照プロジェクトには Godot VMC Tracker が含まれており、パイプラインの診断を目的としています。完全版 ICLC27 ワークスペースの準備済みプロジェクトには、パフォーマンス例で使用する設定済みアバターが含まれています。

アバターの受信ポートとトラッカー名については、[Port Number Setup](#)および [Godot における複数アバタープロジェクトのポート番号設定](#)を参照してください。

インストールを確認する

完全なライブパイプラインを試す前に、次のコマンドを個別に確認します。

```
python3 --version
python3 PipelineApplications/Bunraku0SCEncoder.py --help
python3 PipelineApplications/Bunraku0SCDecoder.py --help
```

SuperCollider では次を確認します。

Bmc.status;

Godot では、選択したプロジェクトがアドオン不足やスクリプトエラーなしで開くことを確認します。

以上でインストールは完了です。小さなユーザーテストを順番に行うには [はじめに](#)へ進み、XR-Animator-BuMoChi-Godot パイプライン全体を起動して接続するには [BuMoChi フルセットアップ手順](#)を使用してください。